

Lab2：批处理系统

本章完成的工作

本章完成了批处理系统的运行和理解：

- 1. 成功运行 ch2 分支，观察多个用户程序自动执行
- 2. 理解"批处理"的含义和历史背景
- 3. 初步接触特权级（U态/S态）的概念
- 4. 分析用户程序是如何被加载和执行的

本章没有编程作业，代码由清华提供，重点是理解批处理系统的工作原理。

一、实验步骤

1.1 进入代码目录并运行

```
cd /桌面/herdream/2025-ucore-riscv-清华/uCore-Tutorial-Code-2025S-ch2（其他同学可根据实际路径）
make user BASE=1 CHAPTER=2
make run
```

注：user 目录需预先从测例仓库下载并放入，详见 Lab0 说明。

1.2 运行结果

```
[rustsbi] RustSBI version 0.3.0-alpha.2, adapting to RISC-V SBI v1.0.0
.-----
| _ \   | | | |   /      | /      || _ \ | | | | | |
| |_) |   | | | |   (----`---| |----`---| |_) || |
|   /    | | | |   \ \    | |    \ \    | _ < | |
|  \| \---.| `--' |.----) |    | | .----) | | |_) || |
|_| | `_.|| \___/ |___/   | | | |___/   |___/ | |
[rustsbi] Implementation      : RustSBI-QEMU Version 0.2.0-alpha.2
[rustsbi] Platform Name       : riscv-virtio,qemu
[rustsbi] Platform SMP        : 1
[rustsbi] Platform Memory     : 0x80000000..0x88000000
[rustsbi] Boot HART           : 0
[rustsbi] Device Tree Region  : 0x87000000..0x87000ef2
[rustsbi] Firmware Address    : 0x80000000
[rustsbi] Supervisor Address  : 0x80200000
[rustsbi] pmp01: 0x00000000..0x80000000 (-wr)
[rustsbi] pmp02: 0x80000000..0x80200000 (---)
[rustsbi] pmp03: 0x80200000..0x88000000 (xwr)
[rustsbi] pmp04: 0x88000000..0x00000000 (-wr)
hello world!
Running ch2b_exit, about to exit with 1234
Hello world from user mode program!
Test hello_world OK!
```

```
Hello! This is app 4: ch2b_myapp
3^10000=5079
3^20000=8202
3^30000=8824
3^40000=5750
3^50000=3824
3^60000=8516
3^70000=2510
3^80000=9379
3^90000=2621
3^100000=2749
Test power OK!
ALL DONE
```

二、学习过程与理解

2.1 初次运行的观察

运行结果让我有些惊讶：内核输出 "hello wrold!" 之后，竟然自动执行了好几个不同的程序（exit、hello_world、myapp、power），而且一个接一个，中间没有任何人工干预。

这和 Lab1 完全不同——Lab1 只是内核自己打印一些信息就结束了，而这里出现了"用户程序"的概念。

2.2 什么是"批处理系统"

指导书开头提到了"批处理系统"这个词，说是计算机早期为了提高效率而发明的。一开始不太理解为什么需要这个东西。

查阅资料后明白了：早期计算机非常昂贵，如果每运行一个程序都要人工操作（装卡片、等输出、换下一个），计算机大部分时间都在"等人"，太浪费了。所以发明了批处理系统——把多个程序打包在一起，让计算机自动一个接一个地执行。

现在看运行结果，确实是这个效果：4个测试程序自动依次执行，不需要人工干预。

2.3 特权级：为什么需要区分内核和用户程序

指导书提到了"特权级"和"U态、S态"，这个概念一开始很抽象。

通过查阅资料和思考，理解到核心问题是：**如果用户程序可以随意执行任何指令，那一个有bug的程序可能会破坏整个系统。**比如用户程序如果能直接操作硬件，写错一个地址就可能把内核的代码覆盖掉。

所以需要把指令分成两类：

- 普通指令：用户程序可以执行
- 特权指令：只有内核才能执行（如访问硬件、修改页表等）

这就是 U态（User）和 S态（Supervisor）的区别。用户程序运行在 U态，想做"危险操作"时，必须通过**系统调用**请求内核帮忙。

2.4 从代码中验证理解

带着这些理解去看代码，很多东西就清晰了。

在 `os/trap.c` 中找到了系统调用的处理入口：

```
void trap_handler(void)
{
    struct trapframe *tf = curr->trapframe;
    uint64 scause = r_scause();

    if (scause == CAUSE_USER_ECALL) {
        // 用户程序通过 ecall 指令触发系统调用
        tf->epc += 4; // 跳过 ecall 指令
        tf->a0 = syscall(); // 处理系统调用，返回值放入 a0
    }
    // ...
}
```

原来 `ecall` 指令就是用户程序"求助"内核的方式！执行这条指令后，CPU 会自动从 U态切换到 S态，然后跳转到内核的 `trap` 处理函数。

2.5 用户程序是怎么被加载的

另一个疑问是：这些用户程序是从哪里来的？怎么"装进"内核的？

在 `os/loader.c` 中找到了答案：用户程序在编译阶段就被打包进内核镜像了！通过 `link_app.s` 这个汇编文件，把用户程序的二进制数据嵌入到内核中。运行时，`run_next_app()` 函数把程序复制到用户地址空间（0x80400000）再执行。

这就解释了为什么 `make user` 要在 `make run` 之前执行——需要先编译用户程序，才能把它们打包进内核。

2.6 回顾 Lab1：现在理解更深了

回头看 Lab1 的启动流程图，现在理解更深了：

```
QEMU → RustSBI(M态) → 内核(S态) → 用户程序(U态)
```

Lab1 只走到了"内核(S态)"这一步，Lab2 才真正进入了"用户程序(U态)"。原来 Lab1 说的"内核运行在 S态"，是为后面的用户态/内核态切换做铺垫的。

三、核心概念总结

3.1 本章涉及的系统调用

刚开始看到"系统调用"这个词，总觉得很高大上很神秘。，在我看来，`ecall` 指令应该是敲内核的门，喊一声"帮个忙"，然后内核根据你传的参数决定帮你做什么。

系统调用	调用号	功能
sys_write	64	向控制台输出字符串
sys_exit	93	退出当前程序，运行下一个

3.2 特权级切换流程

用户程序(U态) --ecall--> trap_handler(S态) --sret--> 用户程序(U态)

四、实验总结

本章完成了以下工作：

- 成功运行 ch2 的批处理系统
- 理解批处理系统的历史背景和设计目的
- 理解特权级机制（U态/S态）的必要性
- 分析 trap.c 中系统调用的处理流程
- 理解用户程序的加载机制

收获：本章最大的收获是理解了"为什么需要区分用户态和内核态"。这不是故意把事情搞复杂，而是为了保护系统不被有问题的用户程序破坏。批处理系统虽然简单，但已经具备了操作系统的核心特征：管理和隔离。

五、验证截图

```
[rustsbi] Platform Memory      : 0x80000000..0x88000000
[rustsbi] Boot HART            : 0
[rustsbi] Device Tree Region   : 0x87000000..0x87000ef2
[rustsbi] Firmware Address     : 0x80000000
[rustsbi] Supervisor Address   : 0x80200000
[rustsbi] pmp01: 0x00000000..0x80000000 (-wr)
[rustsbi] pmp02: 0x80000000..0x80200000 (---)
[rustsbi] pmp03: 0x80200000..0x88000000 (xwr)
[rustsbi] pmp04: 0x88000000..0x00000000 (-wr)
hello world!
Running ch2b_exit, about to exit with 1234
Hello world from user mode program!
Test hello_world OK!
Hello! This is app 4: ch2b_myapp
3^10000=5079
3^20000=8202
3^30000=8824
3^40000=5750
3^50000=3824
3^60000=8516
3^70000=2510
3^80000=9379
3^90000=2621
3^100000=2749
Test power OK!
ALL DONE
hjl@hjl-VMware-Virtual-Platform:~/桌面/herdream/2025-ucore-riscv-清华/uCore-Tutorial-Code-2025S-ch2$
```